

Введение 1 в DOM

1. Объектная модель документа (DOM)

HTML-теги каждого HTML-документа, отображаемого в браузере, составляют иерархическую структуру, которая называется *объектной моделью документа* (**DOM** — **Document Object Model**).

HTML-теги документа называют объектами модели DOM.

Иерархия тегов понимается в том смысле, что если один тег входит в другой, то входящий является дочерним для второго. Например, теги HTML-документа

```
<html>
<head>
<title> Пример текста </title>
</head>
<body>
<h1> Представление документа в виде дерева </h1>
<p>Абзац 1</p>
<p>Абзац 2</p>
</body>
</html>
```

представляются в виде следующей иерархической модели (рис. 1):

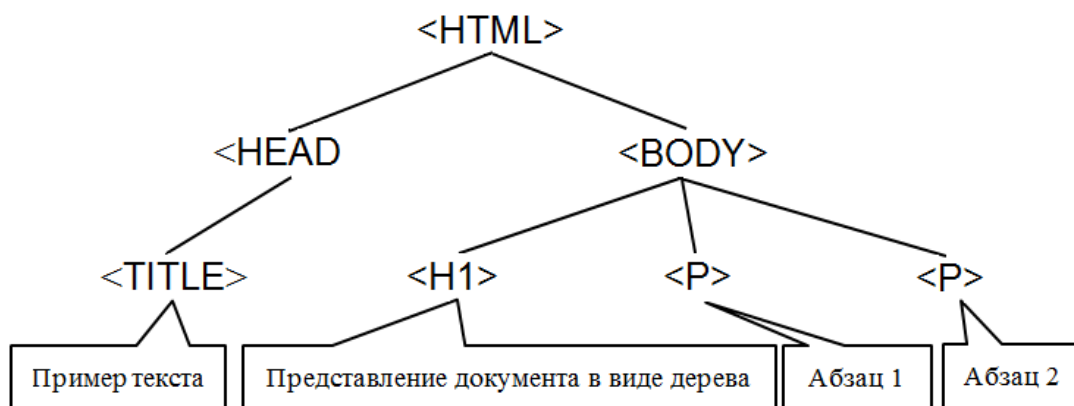


Рис. 1.

Изображение всей страницы задается объектом DOM, который определяется парой тегов `<html></html>`. Дочерними для него являются объекты, задаваемые тегами `<head></head>` и `<body></body>`, и т.д.

2. Атрибуты и стилевые свойства объектов DOM

Свойства любого объекта DOM можно описать с помощью атрибутов и стилевых свойств. Способы описания рассмотрим на примере.

Пример 1.

```
<html>
<head>
```

```
<title>Пример 1</title>
</head>
<body bgcolor="green" text="red">
<p> Данный <b><i>текст является примером</i></b>, который
будет использоваться для знакомства <font face="Arial"
color="yellow"> с приемами записи простейших меток</font>
<i>языка <U>HTML</u></i>.</p>

</body>
</html>
```

запишем текст примера 1, сохраним его в файле
pos1.html

Первый способ описания свойств объектов DOM — это описание с помощью атрибутов. В начальном теге объекта атрибуту присваивают конкретное значение.

Так, в примере 1 объект, задаваемый тегами <body></body>, имеет зеленый цвет фона (запись bgcolor="green" означает, что атрибуту bgcolor присвоено значение green) и красный цвет текста (text="red").

Объект, задаваемый тегами , имеет шрифтовую гарнитуру Arial (face="Arial") и желтый цвет текста (color="yellow").

Рисунок, задаваемый тегом , имеет вертикальный размер 150 пикселей (height=150).

К сожалению, имена одних и тех же атрибутов в разных тегах могут различаться. Например, цвет текста в теге <body> задает атрибут text, а в теге — атрибут color.

Второй способ описания свойств объектов DOM — это описание стилевых свойств.

Стилевые свойства записываются как значение универсального атрибута style (стиль) по следующей форме:

style="имя_свойства1:значение1;имя_свойства2:значение2;"

Например,

style="position:absolute;top:240px;left:100px;color:blue;"

Значение атрибута **style** включает одно или несколько *стилевых правил*, разделенных точками с запятой (список стилевых правил).

В стилевом правиле слева записывается *имя стилевого свойства*, а после двоеточия — *значение стилевого свойства*. В списке и в стилиевых правилах около знаков препинания можно добавлять пробелы.

Список стилиевых правил атрибута `style` обычно называют *каскадной таблицей стилей (CSS — Cascad Style Sheet)*.

Описание атрибута `style` в тегах объектов называют встраиванием таблиц стилей.

Теперь, чтобы строить веб-страницы с использованием каскадных таблиц стилей, достаточно знать имена стилиевых свойств объектов DOM и их возможные значения.

Перечень основных стилиевых свойств приведен в следующем пункте.

3. Основные стилиевые свойства объектов DOM

Имя стилевого свойства	Описание	Примеры значений
Свойства текста		
color	Цвет символов текста	red, green, blue, #ff00aa
font-family	Название гарнитуры шрифта	Helvetica, Serif, Symbol
font-size	Размер шрифта	xx-small, x-small, small, medium, large, x-large, xx-large (относительные размеры шрифта, заданные в браузере); 25px (пиксели), 12pt (пункты), 120% (проценты от высоты шрифта)
font-weight	Насыщенность шрифта	bold (полужирное начертание), bolder (жирное начертание), lighter (светлое начертание), normal (нормальное начертание); 500 (условные единицы от 100 до 900)
font-style	Стиль шрифта	italic (курсив), normal
text-align	Тип горизонтального выравнивания текста	left, right, center, justify (по ширине)
text-decoration	Оформление текста	underline (подчеркивание), line-through (перечеркивание), overline (линия над текстом), none
text-transform	Преобразование текста в заглавные или прописные символы	capitalize (первая буква слова заглавная), lowercase (все буквы строчные), uppercase (все буквы заглавные), none
vertical-align	Выравнивание текста по вертикали относительно ячейки таблицы или окружающего текста	baseline, top, middle, bottom; sub, sup

Имя стилевого свойства	Описание	Примеры значений
text-indent	Отступ первой строки текста	10% (процент от длины блока); <i>метрические значения:</i> 25px (пиксели), 12pt (пункты), 1.5in (дюймы), 1.2cm (сантиметры), 15mm (миллиметры).
word-spacing	Интервал между словами	
letter-spacing	Интервал между символами в словах текста	
line-height	Межстрочный интервал текста	1.5 (полуторный интервал); 10% (процент от высоты шрифта); <i>метрические значения</i> (см. выше).
text-overflow	Параметры видимости текста в блоке, если текст целиком не помещается	clip (текст обрезается по размеру области), ellipsis (текст обрезается и к концу строки добавляется многоточие)
Фон объекта		
background-color	Цвет	white, #0000ff
background-image	Рисунок, фото	В виде ссылки: url('image.gif'), url('http://www.corp.by/ber.jpg')
background-repeat	Повторение рисунка	repeat, repeat-x, repeat-y, no-repeat
Размеры объекта		
width	Ширина блочного объекта или рисунка	<i>метрические значения</i> (см. выше); 40% (процент от ширины родит. объекта).
height	Высота блочного объекта или рисунка	<i>метрические значения</i> (см. выше); 50% (процент от высоты родит. объекта).
Положение в окне браузера		
position	Способ позиционирования объекта в окне браузера	absolute (абсолютное позиционирование — относительно краев окна браузера), relative (относительное позиционирование — относительно исходного положения в окне браузера или краев родительского объекта)
top (должно быть задано значение position)	Расстояние по вертикали сверху (в зависимости от типа позиционирования)	<i>метрические значения</i> (см. выше); 50% (процент от высоты родительского объекта).
left (должно быть задано значение position)	Расстояние по горизонтали слева (в зависимости от типа позиционирования)	
right (должно быть задано значение position)	Расстояние по горизонтали справа (в зависимости от типа позиционирования)	

Имя стилевого свойства	Описание	Примеры значений
	позиционирования)	
Отображение или скрытие объекта		
visibility	отображение или скрытие объекта, причем место, которое объект занимает, остается за ним	visible (объект видим), hidden (объект невидим, полностью прозрачный).
display	отображение или скрытие объекта, причем объект исключается из текста документа	block (отображение), none (скрытие).
z-index (должно быть задано значение position)	отображение при наложении объектов (обычно, чем позднее по тексту описан объект, тем выше он отображается)	<i>целые числа</i> (чем больше число, тем выше объект отображается)
opacity (для IE9+)	прозрачность изображения объекта	<i>число из диапазона [0; 1.0]:</i> 0 (полностью прозрачный), 0.5 (полупрозрачный), 1 (полностью видимый).
filter (для IE6-8)	фильтр для изменения изображения объекта, включая прозрачность	<i>значение для прозрачности:</i> alpha(opacity=50) , где целое число из диапазона [0; 100] — процент видимости.

Универсальность каскадных таблиц стилей состоит в том, что имена стилевых свойств определены однозначно и не зависят от вида тега, в котором они используются.

Например, в примере 1 для цвета текста в теге <BODY> используется атрибут с именем text, а в теге — атрибут с именем color. В таблицах стилей в этом случае всегда используется стилевое свойство с именем color.

Пример 2.

В тексте примера 1 заменим во всех тегах атрибуты описанием стилевых правил в универсальном атрибуте **style**.

В тексте примера 1 меняем номер примера и сохраняем файл под именем pos2.html. В начальном теге <body> удалим атрибуты bgcolor="green" и text="red" и вместо них в соответствии с перечнем свойств запишем команду для атрибута **style** в следующем виде

style="background-color:green;color:red;"

Далее, в начальном теге удалим атрибуты face="Arial" и color="yellow" и добавим описание атрибута style вида

style="font-family:Arial; color:yellow;"

В теге вместо атрибута height=150 записываем команду

style="height:150;"

Поменяем также в теге <title></title> номер примера.

Сохраним файл pos2.html и запустим

Заметим, что использовать таблицы стилей можно при описании практически всех объектов, а атрибуты имеют далеко не все объекты. Да и разнообразных стилевых свойств в таблицах стилей гораздо больше, чем возможных атрибутов в тегах.

Продолжение примера 2.

В теге введем описание цвета текста (синий)

style="color:blue; "

Сохраним текст в файле pos2.html и запустим .

Объект действительно меняет цвет.

Продолжение примера 2.

Добавим в теге в атрибут style еще одно стилевое правило описания фона текста (желтый)

background-color:yellow;

Команда для атрибута style примет вид

style="color:blue;background-color:yellow;"

Текст сохраним в файле pos2.html и запустим

В результате у объекта появляется фон желтого цвета.

Таким образом, таблицы стилей добавляют возможности, которые иным образом реализовать невозможно. Например, можно установить другое положение объекта веб-страницы в окне браузера.

Продолжение примера 2.

В объекте добавим в значение атрибута style три новых стилевых правила для нового расположения объекта в окне браузера

position:absolute; top:250px; left:100px;

Стилевые правила означают, что объект позиционируется относительно верхнего и левого краев окна (position:absolute – абсолютно), от верхнего края 250 пикселей (top – сверху) и от левого края 100 пикселей (left – слева). Команда для атрибута style получает вид

style="color:blue;background-color:yellow;position:absolute; top:250px; left:100px;"

Текст сохраним в файле pos2.html и запустим

В результате на экране появляется довольно интересное изображение.

4. Описание каскадных таблиц стилей с помощью тега <STYLE>

При описании свойств объектов DOM каскадные таблицы стилей могут быть достаточно объемными (см. пример 2). Причем часто у разных объектов таблицы совпадают.

Чтобы не записывать каждый раз в тегах с помощью атрибута `style` длинные таблицы стилевых правил, их описывают и именуют специальным образом.

Описание делается в HTML-тексте в разделе `<head></head>` с помощью парных тегов

`<style></style>`

Пример 3.

В тексте примера 2 опишем в разделе `<style></style>` все таблицы стилей, в которых два и более стилевых правил. Такой вид имеют таблицы в тегах `<body>`, `<b>` и `<font>`.

В разделе `<head></head>` HTML-текста изменяем номер примера и вводим раздел `<style></style>`. В результате получаем:

```
<head>
<title> Пример 3 </title>
<style>
.st1 {background-color:green; color:red;}
.st2 {color:blue;background-color:yellow;position:absolute; top:
250px; left:100px;}
.st3 {font-family:Arial; color:yellow;}
</style>
</head>
```

В начале каждой строки описания после точки записано имя таблицы стилей, а далее в фигурных скобках — список стилевых правил. Это сокращенное описание. Удобнее записывать в развернутом виде, размещая стилевые правила с новой строки каждое. Разбиваем строки таблиц стилей и приводим текст раздела `<style></style>` к виду

```
<head>
<title> Пример 3 </title>
<style>
.st1 {
    background-color:green;
    color:red;
}
.st2 {
    color:blue;
    background-color:yellow;
    position:absolute;
    top:250px;
    left:100px;
}
.st3 {
    font-family:Arial;
    color:yellow;
}
</style>
```

```
</head>
```

Чтобы использовать заготовленные таблицы стилей в тегах используется универсальный атрибут **class**, который записывается по схеме

```
class="имя_таблицы"
```

В части `<body></body>` удаляем в тегах ненужные атрибуты **style**, а вместо них записываем команды присваивания атрибуту **class** имени соответствующей таблицы (без точки!):

```
<body class="st1">
```

```
<b class="st2">
```

```
<font class="st3">
```

Сохраним текст в файле `pos3.html` и запустим. Если все сделано правильно, то изображение не поменяется.

5. Имя таблицы CSS как селектор

Технология CSS позволяет еще более сократить записи при введении таблиц стилей. Мы ввели уникальные имена таблиц стилей для тегов `<body>`, `<b>` и `<font>`. Уникальные имена в описании всегда начинаются с точки.

Но так как теги `<b>` и `<font>` вложены в тег `<body>`, то для именования таблиц стилей вложенных тегов используют модель DOM (без уникальных имен).

Например, если для некоторого тега задана таблица стилей с именем **tab**, а в него вложен тег `<div>`, тогда для вложенного тега таблица стилей автоматически получает составное имя **tab div** (с пробелом). Таким образом:

Имя таблицы стилей является ***селектором***, избирающим соответствующий объект модели DOM из всех возможных.

Пример 4.

В примере 3 заменим, где это возможно, уникальные имена таблиц стилей на имена-селекторы.

Загружаем файл `pos3.html`, меняем номер примера на «4» и сохраняем файл под именем `pos4.html`.

Тег `<body>` имеет таблицу стилей с уникальным именем **st1**, тогда таблица стилей вложенного тега `<b>` имеет имя **st1 b** (с пробелом), а таблица стилей вложенного тега `<font>` имеет имя **st1 font** (с пробелом).

Меняем имена двух таблиц стилей в разделе `<style></style>` и получаем

```
<style>
.st1 {
  background-color:green;
  color:red;
}
.st1 b {
  color:blue;
  background-color:yellow;
  position:absolute;
```

```
top:250px;
left:100px;
}
.st1 font {
font-family:Arial;
color:yellow;
}
</style>
```

Теперь осталось в первых тегах `<b>` и `<font>` удалить команды присвоения значений атрибуту `class`. Сохраним файл `pos4.html` и запускаем

6. События

Понятие "*событие*" является понятием динамического HTML. События могут инициироваться пользователем или процессом, происходящим в компьютере. События имеют определенные имена, в частности:

`onkeydown` — пользователь нажал любую клавишу клавиатуры, но не отпустил;

`onkeypress` — пользователь нажал и отпустил любую клавишу клавиатуры;

`onkeyup` — пользователь отпустил нажатую клавишу клавиатуры;

`onhelp` — пользователь нажал клавишу F1 клавиатуры;

`onclick` — пользователь выполнил щелчок левой клавишей мыши по объекту HTML;

`ondblclick` — пользователь выполнил двойной щелчок левой клавишей мыши по объекту;

`onmousedown` — пользователь нажал левую кнопку мыши;

`onmouseup` — пользователь отпустил левую кнопку мыши;

`onmouseover` — пользователь впервые ввел указатель мыши в границы объекта;

`onmousemove` — пользователь двигает указатель мыши над объектом;

`onmouseout` — пользователь вывел указатель мыши за границы объекта;

`onload` — завершилась загрузка объекта для отображения в браузере;

`onresize` — пользователем или программой поменялись размеры окна браузера.

В HTML есть и другие события, которые используются не так часто, как перечисленные.

7. Как связать объект и событие средствами HTML

Чтобы объект DOM стал реагировать на определенное событие, в начальный тег этого объекта вносится атрибут вида

`имя_события = "func()"`

где `func()` — имя функции языка JavaScript.

Как только с объектом происходит событие с таким именем, выполняется функция, записанная в равенстве справа.

Пример 5.

Допишем в тексте предыдущего примера в теге `<font>` атрибут `onclick = "alert('событие 1')"`

Получим

`<font onclick = "alert('событие 1')">` с приемами записи простейших меток `</font>`

Записанный атрибут означает, что при щелчке указателя мыши по изображению этого текста (событие `onclick`), будет выполнена команда языка JavaScript `alert('событие 1')`, которая выведет на экран диалоговое окно с надписью "событие 1".

Внимание!!! Обращаем внимание на то, что в записи атрибута использованы кавычки двух типов: двойные обрамляют команду языка, а одинарные — текстовую константу.

Текст примера сохраняем в файле `pos5.html` и запускаем его.

Так, по сути, каждому объекту DOM можно приписать реакцию на событие. Более того, одному объекту можно приписать реакцию на несколько событий. Обычно таким объектом является тег `<body>`.

Продолжение примера 5.

Допишем в тег `<body>` еще две реакции на события:

`<body class="st1" ondblclick = "alert('событие 2')"`
`onkeypress = "alert('событие 3')">`

Текст продолжения примера сохраняем в файле `pos5.html` и запускаем его. Тегу `<body>` на экране соответствует вся внутренность окна браузера.

(!!! Пример 5 после выполнения показать преподавателю!!!10)

Замечание. Включение в тег атрибута вида `ondblclick = "alert('событие 2')"` обычно называют включением "прослушивания" события `ondblclick` в теге.

Событие всегда вызывает заданную реакцию (в данном случае выполнение команды `alert('событие 2')`). То же самое говорят и об атрибутах для других событий: *запись атрибута для события — это включение прослушивания события.*

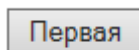
8. Формы и кнопки

В языке HTML для создания интерактивных веб-страниц предусмотрено несколько специальных объектов. Главный из них — это объект Form (форма), который задается парными тегами `<form></form>` и обычно включает в себе несколько одиночных тегов `<input>`.

Тег в виде `<input type="button">` задает на экране изображение виртуальной прямоугольной кнопки. Атрибут `type` имеет значение `button` — по-английски «кнопка». Текст, который отображается на кнопке, задается в теге вторым атрибутом `value` с текстовым значением. Например, запись

```
<input type="button" value="Первая">
```

выводит кнопку вида



Следует обратить внимание на то, что кавычки в редакторе в текст должны вводиться с помощью латинской раскладки клавиатуры, в противном случае они не воспринимаются браузером.

В форму также часто включают:

- текстовое поле (тег `<input type="text">`, обычно предназначено для ввода символов с помощью клавиатуры). Вид поля:



- переключатели (тег `<input type="radio">`, используются, когда следует выбрать один вариант из нескольких предложенных). Например:

☐ Пиво ☐ Чай ☐ Кофе

- флажки (тег `<input type="checkbox">`, позволяют выбрать более одного варианта из предложенных). Например:

☐ Пиво ☐ Чай ☐ Кофе

- кнопки-изображения (тег `<input type="image">`), возможный вид:



Пример 6. Создадим изображение на зеленом фоне, в котором ниже текста "Пример использования формы с кнопками" желтого цвета расположена форма с тремя кнопками, имеющими названия "Первая", "Вторая", "Третья".

```
<html>

<head>
  <title> Пример 6  </title>
</head>
<body style="background-color:green">
  <p style="color:yellow">Пример использования формы с
кнопками</p>
  <form>
    <input type="button" value = "Первая">
    <input type="button" value = "Вторая">
    <input type="button" value = "Третья">
```

```
</form>
</body>
</html>
```

Запишем текст `pos6.html` и сохраним его в файле **pos6.html** и запустим

Следует заметить, что, хотя никакой интерактивной реакции мы не задавали, но событие **onmouseover** вызывает изменение в изображении кнопки (появляется цветной внутренний фон или контур). А щелчок по кнопке (событие **onclick**) вызывает на кнопке постоянный внутренний контур другого цвета (пунктир). Это особенность объекта **<input type="button">**.

Пример 7.

В условиях примера 6 добавим к каждой кнопке реакцию на щелчок указателем мыши (событие **onclick**). Для этого в файле **pos6.html** допишем в теги **<INPUT>** по третьей команде вида

```
onclick = "alert()"
```

В качестве аргумента команды `alert()` будем записывать текст с подтверждением нажатия кнопки. Теги **<input>** получат вид

```
<input type="button" value="Первая" onclick="alert('Вы нажали
первую кнопку')">
<input type="button" value="Вторая" onclick="alert('Вы нажали
вторую кнопку')">
<input type="button" value="Третья" onclick="alert('Вы нажали
третью кнопку')">
```

Снова заметим, что функция `alert()` записана в двойных кавычках, а текст аргумента — в одинарных. Одни кавычки нужны по правилам записи функции как реакции на событие, а другие — по правилу записи символьных значений.

Поменяем в теге **<title></title>** номер примера.

Сохраним текст в файле **pos7.html** и запустим его. Теперь можно проверить реакцию веб-страницы на щелчки мышью по кнопкам.

9. Использование функции пользователя

В примере 7 в качестве реакции на событие была использована известная команда-функция `alert()` языка JavaScript. Теперь для организации реакции воспользуемся специальными функциями, которые сами и создадим.

Пример 8.

Изменим пример 7 и запишем реакции на события с использованием функций пользователя.

Для этого внесем изменения в текст файла **pos7.html**. Вызываем его в редактор, меняем в теге **<title> </title>** номер примера и сразу сохраняем под именем **pos8.html**.

Далее в теге первой кнопки **<input type="button">** в атрибуте

```
onclick = "alert('Вы нажали первую кнопку')"
```

заменяем текст "alert('Вы нажали первую кнопку')" на "func1()".

Аналогично, в теге второй кнопки в атрибуте

```
onclick = "alert('Вы нажали вторую кнопку')"
```

заменяем текст "alert('Вы нажали вторую кнопку')" на "func2()".

И, наконец, в теге третьей кнопки в атрибуте

```
onclick = "alert('Вы нажали третью кнопку')"
```

заменяем текст "alert('Вы нажали третью кнопку')" на "func3()".

Новые функции func1(), func2() и func3() надо описать на языке JavaScript в разделе <head></head>.

Для этого, перед конечным тегом </head> вставляем пару тегов

```
<script>  
  
</script>
```

Между этими тегами вставляем из файла Help_JS.htm три конструкции функции пользователя (первого вида):

```
function    (    ) {  
  
}
```

Соответственно, даем функциям имена func1, func2 и func3.

А теперь в теле первой функции func1() записываем команду (без внешних кавычек)

```
alert('Вы нажали первую кнопку');
```

В теле второй функции func2() — команду

```
alert('Вы нажали вторую кнопку');
```

В теле третьей функции func3() — команду

```
alert('Вы нажали третью кнопку');
```

Сохраняем файл pos8.html и запускаем его.

Реакции на события файла pos7.html должны сохраниться.

(!!! Пример 8 после выполнения показать преподавателю!!! 11)